

Módulo 8 – Capítulo 05 – Sección 01

Guía de selección y vLLM: PagedAttention y continuous batching para producción GPU

Antes de profundizar en los motores de serving de producción, es necesario establecer cuándo corresponde usar cada herramienta disponible. Las cuatro opciones principales del ecosistema de serving de LLMs tienen casos de uso primarios distintos y elegir el motor incorrecto resulta en complejidad innecesaria o rendimiento subóptimo:

Escenario	Motor recomendado
Desarrollo local / prototipado	Ollama + llama.cpp (Cap. 3)
Producción GPU, equipo pequeño, un tipo de modelo	vLLM
Producción GPU, múltiples tipos de modelos (embeddings + LLM + clasificadores)	Triton Inference Server con backends específicos
Máxima eficiencia en NVIDIA, costo por token crítico	TRT-LLM

Esta guía evita que el lector profundice en el motor incorrecto para su caso de uso. Para la mayoría de los equipos que despliegan un único LLM en GPU para servir usuarios en producción, **vLLM es la elección correcta**.

vLLM es el motor de serving de LLMs de mayor adopción en producción GPU, desarrollado por el Sky Computing Lab de UC Berkeley, que resuelve el problema fundamental de la gestión ineficiente del KV cache mediante PagedAttention. El problema que vLLM ataca es concreto: los sistemas de inferencia tradicionales pre-reservan una región contigua de VRAM para el KV cache de cada request basada en la longitud máxima de contexto configurada, independientemente de si la request real usa esa longitud completa. Si configuras un contexto máximo de 8192 tokens y recibes peticiones que usan en promedio 1024 tokens, el 87% de la VRAM reservada para KV cache está siendo desperdiciada en fragmentación interna. Con

1.000 requests concurrentes posibles pero solo 100 activas en promedio, 900 "slots" de VRAM están vacíos pero bloqueados.

PagedAttention, el mecanismo central de vLLM, resuelve este desperdicio aplicando el mismo principio de paginación de memoria virtual que los sistemas operativos usan para la RAM física: divide el KV cache en bloques de tamaño fijo (páginas de 16 tokens por defecto) y los asigna dinámicamente desde un pool centralizado solo cuando son necesarios. Cuando una request termina, sus bloques de KV cache se liberan de vuelta al pool y quedan disponibles para nuevas requests sin necesidad de defragmentación. El resultado medido es que la fragmentación interna cae de 60-80% en implementaciones tradicionales a menos del 4% con PagedAttention, permitiendo servir 3-4x más peticiones simultáneas con el mismo hardware.

El continuous batching (también llamado iteration-level scheduling) complementa PagedAttention en la dimensión del tiempo. En sistemas de batching tradicionales, un batch se forma al inicio, se procesa hasta que todas las requests del batch terminan, y solo entonces se acepta el siguiente batch. Si el batch tiene una request que genera 2.000 tokens (larga) y otras nueve que generan 50 tokens (cortas), las nueve requests cortas terminan en fracciones del tiempo total, pero la GPU permanece "esperando" a la request larga con los slots de las requests cortas vacíos. Con continuous batching, en cada iteración de decode vLLM puede incorporar nuevas requests al batch activo: tan pronto como una request corta termina, su slot en el batch se rellena con la siguiente request en la cola, manteniendo la GPU ocupada de forma continua.

Componentes técnicos de vLLM

- **PagedAttention:** KV cache dividido en bloques de 16 tokens asignados dinámicamente; reduce fragmentación interna a <4%; permite prefix caching compartiendo bloques entre requests con el mismo prefijo.
- **Continuous batching:** nuevas requests se incorporan al batch activo en cada iteración de decode; elimina el "bubble time" de GPU idle esperando que el batch completo termine.
- **Prefix caching:** cuando múltiples requests comparten el mismo system prompt o prefijo largo, vLLM comparte los bloques del KV cache entre todas; reduce el compute de prefill y el uso de VRAM.
- **Tensor parallelism:** `tensor_parallel_size=N` distribuye el modelo entre N GPUs con sharding de pesos; pipeline parallelism para modelos que no caben en multi-GPU tensor-parallel.

- **Gestión de memoria dinámica:** copy-on-write para beam search; garbage collector de bloques al finalizar requests; uso de VRAM predecible y máximo por diseño, sin memory leaks.

Nota del Arquitecto: *PagedAttention y continuous batching no son optimizaciones opcionales que se activan cuando se busca rendimiento extra —son la razón fundamental por la que vLLM existe. La diferencia entre Hugging Face Transformers en modo básico y vLLM con el mismo hardware puede ser de 5-10x en throughput agregado. Para cualquier sistema que sirva a más de 2-3 usuarios concurrentes, vLLM no es un lujo sino un requisito.*

La arquitectura interna de vLLM y su mecanismo de PagedAttention establecen el fundamento sobre el que se construye todo el sistema de serving de producción. La sección siguiente detalla cómo configurar y desplegar vLLM como servidor de producción con API compatible con OpenAI.

Módulo 8 – Capítulo 05 – Sección 02

OpenAI-compatible API con vLLM: despliegue listo para producción

El servidor HTTP integrado de vLLM convierte el motor de inferencia en un servicio de producción con una sola línea de comando. El comando de inicio básico es `vllm serve meta-llama/Llama-3.1-8B-Instruct --host 0.0.0.0 --port 8000`, que descarga el modelo desde Hugging Face Hub si no está en caché local, lo carga en GPU usando PagedAttention, y levanta un servidor HTTP que expone los endpoints `/v1/chat/completions`, `/v1/completions`, `/v1/embeddings` y `/v1/models` con el mismo contrato de request/response que la API oficial de OpenAI.

El parámetro `--gpu-memory-utilization` es la palanca principal de configuración del pool de KV cache: con `--gpu-memory-utilization 0.90`, vLLM reserva el 90% de la VRAM disponible para el KV cache pool, dejando el 10% restante para el overhead del framework y los buffers de activación. Este valor debe calibrarse según el patrón de uso: incrementarlo aumenta el throughput de requests concurrentes (más bloques disponibles en el pool) pero puede causar OOM errors si hay peticiones con contextos excepcionalmente largos; reducirlo da más margen de seguridad pero desperdicia VRAM disponible. Para producción, comenzar con `0.85` y ajustar basándose en las métricas de `vllm:gpu_cache_usage_perc` expuestas en el endpoint `/metrics`.

La configuración de `--max-model-len` determina el contexto máximo que el servidor puede aceptar: requests con más tokens que este valor son rechazadas con un error HTTP 400 antes de intentar procesarlas, evitando el OOM que resultaría de intentar asignar bloques de KV cache para un contexto que no cabe en el pool. Para modelos con ventanas de contexto largas (Llama 3.1 con 128K tokens), es importante limitar este valor al contexto máximo real que el producto necesita: un `--max-model-len 8192` es suficiente para la mayoría de las aplicaciones conversacionales y reduce significativamente el pool de KV cache necesario frente a 128K.

La cuantización de modelos pre-cuantizados es compatible con vLLM mediante flags específicos: `--quantization awq` para modelos en formato AWQ, `--quantization gptq` para

modelos GPTQ, y `--quantization fp8` para modelos en FP8 nativo en GPUs H100. Para cuantización dinámica sin modelo pre-cuantizado, `--quantization bitsandbytes` aplica INT8 en tiempo de carga, aunque con menor eficiencia que los modelos pre-cuantizados. Los structured outputs, esenciales para aplicaciones que requieren respuestas en JSON con schemas específicos, se habilitan con `--guided-decoding-backend xgrammar`: este parámetro activa la generación guiada por gramática que garantiza que la respuesta del modelo sea siempre JSON válido según el schema especificado en el campo `response_format` de la petición.

Para producción de alta disponibilidad, vLLM se despliega en Kubernetes como un Deployment con `replicas: N`, cada pod con resources `limits: nvidia.com/gpu: 1` (o más para multi-GPU), y un Service de tipo ClusterIP o LoadBalancer. Los endpoints `/health` y `/v1/models` sirven como liveness y readiness probes: vLLM solo responde con HTTP 200 en el endpoint `/health` cuando el modelo está completamente cargado y el servidor está listo para procesar peticiones, evitando que el balanceador de carga envíe tráfico a pods en proceso de inicio de carga del modelo.

Configuración de producción de vLLM

- **Parámetros críticos de memoria:** `--max-model-len` limita el contexto máximo por request; `--gpu-memory-utilization` determina el porcentaje de VRAM para el KV cache pool; `--max-num-seqs` limita las requests concurrentes máximas.
- **Cuantización en vLLM:** `--quantization awq/gptq` para modelos pre-cuantizados; `--quantization fp8` en H100; `--quantization bitsandbytes` para cuantización dinámica INT8.
- **Structured outputs:** `--guided-decoding-backend xgrammar` habilita JSON Schema-guided generation; compatible con el parámetro `response_format` del SDK de OpenAI.
- **Health checks:** `/health` devuelve HTTP 200 solo cuando el modelo está completamente cargado; `/v1/models` lista los modelos disponibles; ambos útiles como Kubernetes probes.
- **Métricas Prometheus:** `/metrics` expone `vllm:num_requests_running`, `vllm:gpu_cache_usage_perc`, `vllm:time_to_first_token_seconds` y `vllm:request_success_total` para dashboards y alertas.

Nota del Arquitecto: La configuración más crítica para evitar problemas en producción es `--max-model-len`. Usarla con el valor máximo soportado por el

modelo (128K para Llama 3.1) parece tentador pero desperdicia VRAM en un pool de KV cache que nunca se llenará si los usuarios del producto rara vez usan más de 8K tokens. Mide el percentil 99 de la longitud de contexto real de tus usuarios y usa ese número como `--max-model-len`; recuperarás VRAM que se traduce directamente en más requests concurrentes atendidas.

La configuración detallada de vLLM para producción incluye docenas de parámetros adicionales para casos especiales, pero los presentados en esta sección cubren el 90% de los escenarios de despliegue de producción. La sección siguiente presenta NVIDIA Triton Inference Server, el motor correcto para el 10% restante de casos donde se necesita servir múltiples tipos de modelos desde una única infraestructura.

Módulo 8 – Capítulo 05 – Sección 03

NVIDIA Triton Inference Server: serving de múltiples modelos y formatos

La tabla de selección de la Sección 01 estableció que Triton Inference Server es el motor correcto cuando se necesita servir simultáneamente múltiples tipos de modelos desde una única infraestructura GPU: no solo LLMs autoregresivos como vLLM sirve con excelencia, sino también modelos de embeddings ONNX, clasificadores de imágenes TensorRT y rerankers de texto en el mismo servidor, con gestión centralizada de recursos de GPU y batching automático para cada tipo de modelo. Confundir Triton con una alternativa a vLLM para LLMs es un error frecuente: son herramientas con propósitos complementarios, no intercambiables.

NVIDIA Triton Inference Server es una plataforma de serving de modelos empresarial que soporta múltiples frameworks de ML (TensorFlow, PyTorch, ONNX Runtime, TensorRT, scikit-learn via FIL) en un único servidor, exponiendo todos los modelos a través de una API gRPC y REST unificada con endpoints `/v2/models/{model}/infer` e `/v2/models/{model}/generate`. La característica diferenciadora de Triton respecto a vLLM es la capacidad de gestionar heterogeneidad de modelos: en una aplicación de búsqueda semántica que usa un modelo de embeddings para vectorizar consultas, un reranker para ordenar resultados y un LLM para generar resúmenes, Triton puede servir los tres modelos desde el mismo servidor con recursos de GPU asignados por modelo y batching adaptado a las características de cada uno.

El model repository de Triton sigue una estructura de directorio estricta que define el comportamiento de cada modelo. Un directorio `text-embeddings/` con un archivo `config.pbtxt` y un subdirectorio `1/` conteniendo el archivo del modelo (ONNX, TensorRT engine, etc.) es suficiente para que Triton cargue y sirva el modelo de embeddings. El `config.pbtxt` configura el batching máximo (`max_batch_size: 32`), el número de instancias del modelo (`instance_group { kind: KIND_GPU count: 1 }`), los tensores de entrada y salida, y la política de dynamic batching que agrupa requests individuales en batches automáticamente dentro de ventanas de tiempo configurables con

`max_queue_delay_microseconds`. Para el LLM del mismo sistema, Triton usaría el backend TensorRT-LLM (presentado en la siguiente sección) con su propio `config.pbtxt` que especifica los paths a los engines compilados.

El dynamic batching de Triton es especialmente valioso para modelos de embeddings y clasificadores que tienen operaciones compute-bound (no memory-bound como el decode de LLMs): agrupa 32-128 peticiones individuales en un único batch que se procesa en paralelo en la GPU, maximizando la utilización de los Tensor Cores y reduciendo el overhead por request en un orden de magnitud respecto a procesar cada petición individualmente. Para LLMs autoregresivos, el dynamic batching de Triton no es tan efectivo como el continuous batching de vLLM, razón por la que el backend TensorRT-LLM para LLMs implementa su propio executor con inflight batching independiente del dynamic batcher general de Triton.

Los model ensembles de Triton permiten definir pipelines de modelos en `config.pbtxt` donde la salida de un modelo alimenta automáticamente la entrada del siguiente, todo servido como una única llamada API desde el cliente: un ensemble `query-pipeline` podría encadenar un preprocesador de texto + el modelo de embeddings + el reranker, con Triton orquestando la transferencia de tensores entre modelos en VRAM sin volver al cliente entre pasos. Esta capacidad de pipelines multi-modelo es imposible de replicar con vLLM en un escenario de producción que necesite múltiples tipos de modelos.

Componentes de Triton Inference Server

- **Model repository:** directorio estructurado con versiones numeradas; Triton monitorea cambios y carga/descarga modelos automáticamente sin reiniciar el servidor.
- **Dynamic batching:** agrupa requests en batches dentro de ventanas de tiempo configurables; ideal para modelos compute-bound (embeddings, clasificadores); no sustituye al inflight batching de LLMs.
- **Model ensembles:** pipelines multi-modelo definidos en `config.pbtxt`; transferencia de tensores en VRAM entre modelos sin volver al cliente; útil para pipelines RAG completos.
- **Backend TensorRT-LLM:** integra TRT-LLM como backend especializado para LLMs; combina optimizaciones de TensorRT con capacidades de batching de Triton.
- **Métricas y trazas:** `/metrics` con latencia por modelo, throughput y uso de GPU; soporte OpenTelemetry tracing con `--trace-config triton,rate=100`.

Nota del Arquitecto: *En sistemas de producción que sirven solo un LLM, Triton añade complejidad operativa sin beneficio técnico frente a vLLM. La señal correcta para considerar Triton es cuando el sistema necesita servir tres o más tipos de modelos distintos (LLM + embeddings + clasificador + reranker) y la gestión de cuatro instancias separadas de distintos motores de serving se vuelve un problema operativo real. Si estás en esa situación, Triton con backends especializados es la arquitectura correcta.*

Triton cierra el espacio de casos de uso de serving empresarial multi-modelo. La siguiente sección presenta TensorRT-LLM, el motor que maximiza la eficiencia del LLM específicamente para hardware NVIDIA cuando el costo por token es la métrica de negocio más importante.

Módulo 8 – Capítulo 05 – Sección 04

TensorRT-LLM: optimización de inferencia para GPUs NVIDIA

Cuando vLLM con sus optimizaciones de PagedAttention y continuous batching no es suficiente para satisfacer los requisitos de throughput o costo por token de una operación de LLM a escala, TensorRT-LLM (TRT-LLM) de NVIDIA es el siguiente nivel de optimización disponible. La diferencia fundamental entre vLLM y TRT-LLM no es de algoritmos de gestión de KV cache —ambos tienen implementaciones eficientes— sino de la profundidad de la optimización del hardware: vLLM opera sobre el modelo en su forma de PyTorch estándar con optimizaciones en tiempo de ejecución; TRT-LLM compila el modelo ahead-of-time a un engine específico para la GPU objetivo, eliminando toda la abstracción de PyTorch y generando código de máquina directamente optimizado para los Tensor Cores y el layout de SRAM de esa GPU específica.

El proceso de uso de TRT-LLM tiene dos fases separadas. La primera es la compilación, que transforma el modelo descargado de Hugging Face en un engine TensorRT para la GPU disponible. El proceso completo involucra dos pasos: primero, `python convert_checkpoint.py --model_dir <ckpt_dir> --output_dir <checkpoint_dir> --dtype bfloat16` convierte los pesos al formato interno de TRT-LLM; segundo, `trtllm-build --checkpoint-dir <checkpoint_dir> --output-dir <engine_dir> --gemm-plugin float16 --max-batch-size 32 --max-input-len 2048 --max-output-len 512` compila el engine. Este proceso tarda entre 30 y 90 minutos para modelos de 7B-70B y produce un archivo `.engine` que solo funciona en la GPU exacta en que fue compilado, con los parámetros de forma exactos especificados en el build.

La segunda fase es la inferencia, donde el engine compilado se ejecuta con el executor de TRT-LLM a través de la integración con Triton usando el backend `tensorrtllm_backend`. Una vez compilado, el engine ejecuta las mismas operaciones que vLLM pero con una eficiencia dramáticamente mayor: en H100 con precisión FP8 nativa (activada con `--use-fp8-context-fmha` durante la compilación), TRT-LLM logra throughput entre 2x y 3.5x superior a vLLM en BF16 para cargas de alta concurrencia en modelos de 7B-70B. Esta mejora proviene de la fusión de kernels (múltiples operaciones de PyTorch que se ejecutan como kernels CUDA

separados se funden en un único kernel optimizado), de la eliminación del overhead de JIT compilation de PyTorch, y del uso de las instrucciones de hardware específicas más eficientes para cada operación en la GPU compilada.

La restricción más importante de TRT-LLM es la especificidad del engine compilado. El mismo modelo compilado en una A100 con `max-batch-size 32` no funciona en una H100, no puede procesar batches de 64 requests, y no puede aceptar inputs más largos que `max-input-len 2048`. Cada cambio en cualquiera de estos parámetros requiere recompilación completa. Esto convierte el proceso de despliegue en una operación más rígida que vLLM: actualizar el modelo base implica recompilación de 30-90 minutos, escalar el batch máximo para un pico de tráfico requiere tener un engine pre-compilado con el nuevo tamaño, y migrar entre tipos de GPU requiere recompilar para cada uno.

El análisis costo-beneficio de TRT-LLM vs vLLM es claro: para organizaciones donde el throughput por GPU-hora determina directamente la rentabilidad del producto (servicios de IA que cobran por token y tienen márgenes ajustados), el 2-3x de mejora de TRT-LLM se traduce directamente en 2-3x más ingresos por unidad de hardware, amortizando el costo operativo adicional de compilación y la rigidez del despliegue en poco tiempo.

Componentes técnicos de TRT-LLM

- **Proceso de build:** conversión de checkpoint con `convert_checkpoint.py` seguida de compilación con `trtllm-build`; 30-90 minutos para modelos de 7B-70B; engine específico para GPU y parámetros de forma.
- **Plugins optimizados:** `--gemm-plugin float16/bfloat16/float8` activa kernels GEMM optimizados de cuBLAS-LT; `--gpt-attention-plugin` activa Flash Attention y KV cache paginado; `--remove-input-padding` elimina padding en batches con longitudes variables.
- **Precisión FP8 en H100:** `--use-fp8-context-fmha` con calibración previa; incrementa throughput 40-80% respecto a FP16 con degradación de calidad mínima en modelos de 7B+.
- **Integración con Triton:** engines desplegados via backend `tensorrtllm_backend`; configuración de `config.pbtxt` para rutas a engines, instancias y parámetros del executor.
- **Speculative decoding:** soporte para Medusa heads, Eagle drafts y modelos draft separados; aceleración de 2-4x en generación dependiendo del dominio.

Nota del Arquitecto: *TRT-LLM es la herramienta correcta cuando ya tienes vLLM en producción, has medido el throughput real con las métricas de Prometheus, y calculas que el 2-3x de mejora de TRT-LLM recupera su costo de implementación (tiempo de ingeniero + rigidez operativa) en menos de 3 meses. Sin ese análisis de ROI, la complejidad adicional no se justifica: vLLM con prefix caching y chunked prefill ya resuelve el 90% de los problemas de throughput.*

TRT-LLM cierra el espectro de motores de serving para máxima eficiencia en NVIDIA. La sección siguiente establece el lenguaje común de métricas (TTFT, TBT, throughput) que permite comparar objetivamente el rendimiento de cualquier motor de serving.

Módulo 8 – Capítulo 05 – Sección 05

Métricas de serving: throughput (tokens/s), latencia TTFT y latencia TBT

Comparar motores de serving, validar que la configuración elegida cumple los SLOs del producto, o diagnosticar problemas de rendimiento en producción requiere un vocabulario de métricas preciso. El error más frecuente es usar el "throughput" como métrica única, cuando en realidad el rendimiento de un sistema de serving de LLMs se describe necesariamente con al menos tres dimensiones ortogonales: el tiempo hasta el primer token (TTFT), el tiempo entre tokens (TBT) y el throughput agregado del sistema. Cada una importa en contextos distintos y se optimiza con técnicas distintas.

El Time To First Token (TTFT) es la métrica de latencia más perceptible para el usuario final en aplicaciones con streaming: mide el tiempo desde que el cliente envía la petición hasta que recibe el primer token de la respuesta. El TTFT incluye el tiempo de cola en el servidor (cuánto espera la petición antes de que el motor la comience a procesar), el tiempo de prefill (procesar todos los tokens del prompt de entrada en la GPU, que crece linealmente con la longitud del input), y el overhead del sistema (serialización, red interna del datacenter). Para aplicaciones interactivas donde el usuario está esperando activamente —asistentes de chat, autocompletado de código, herramientas de búsqueda conversacional— un TTFT inferior a 200-300ms se percibe como "inmediato"; un TTFT de 1-2 segundos se percibe como lento pero tolerable; un TTFT superior a 3 segundos produce abandono. El prefill chunked de vLLM (`--enable-chunked-prefill`) divide prompts largos en chunks que se procesan intercalados con pasos de decode de otras requests, evitando que un prompt de 10.000 tokens bloquee completamente el servidor durante el segundo o más que tarda su prefill.

El Time Between Tokens (TBT), también llamado inter-token latency, mide el tiempo entre tokens consecutivos durante la fase de decode. Este es el parámetro que determina la suavidad del streaming desde la perspectiva del usuario: si el texto aparece en pantalla a 20+ tokens/s ($TBT \leq 50ms$), la experiencia es de texto que "fluye" naturalmente. Si el TBT supera los 80-100ms, el usuario percibe "pasos" o "saltos" en el texto que aparece, degradando la experiencia. El TBT de una request individual aumenta cuando el sistema sirve más requests

concurrentes con continuous batching, porque el motor dedica más ciclos por iteración a las múltiples requests activas. Definir el TBT-P95 objetivo antes de seleccionar el motor de serving determina el número máximo de requests concurrentes que el sistema puede soportar sin degradar la experiencia.

El throughput agregado del sistema, medido en tokens por segundo totales para todos los usuarios simultáneos, es la métrica económica: determina cuántos tokens puede generar el sistema por hora, lo que directamente define el costo por millón de tokens y la capacidad máxima de usuarios del producto. Un A100 con vLLM en modo batch puede generar 2.000-3.000 tokens/s en agregado para Llama 3 8B; un H100 con TRT-LLM puede superar 6.000-8.000 tokens/s. La diferencia de throughput entre vLLM y TRT-LLM justifica el costo adicional de compilación cuando la factura de infraestructura es significativa.

Los percentiles P50, P95 y P99 son críticos para definir los SLOs correctamente. El promedio de TTFT oculta los valores extremos que afectan al 5% o 1% de los usuarios, que en un sistema con millones de usuarios representa decenas de miles de personas con experiencias degradadas. Los SLOs de producción se definen siempre sobre percentiles: "TTFT P95 ≤ 500ms" es un objetivo técnicamente verificable; "TTFT promedio ≤ 300ms" puede cumplirse con un sistema que tiene P99 de 5 segundos.

Métricas clave de serving de LLMs

- **TTFT (Time To First Token):** latencia de red + tiempo en cola + tiempo de prefill; crece con la longitud del input; chunked prefill reduce su variabilidad; SLO típico <300ms P95 para aplicaciones interactivas.
- **TBT (Time Between Tokens):** tiempo de una iteración de decode; aumenta con el número de requests concurrentes en continuous batching; SLO típico <50ms P95 para streaming fluido.
- **Throughput agregado (tokens/s):** métrica de eficiencia de infraestructura; A100 con vLLM: 2.000-3.000 tokens/s; H100 con TRT-LLM: >6.000 tokens/s.
- **E2E latency:** suma de TTFT + (TBT × output_tokens); relevante para aplicaciones que necesitan la respuesta completa antes de procesarla (clasificación, extracción de datos).
- **P50/P95/P99:** los percentiles capturan la distribución real de latencias; SLOs de producción se definen sobre P95 y P99, no sobre el promedio.

Nota del Arquitecto: El primer paso en cualquier proyecto de optimización de serving es medir los percentiles reales con tráfico representativo —no con requests

individuales en aislamiento. La mayoría de los problemas de rendimiento en producción (contención de memoria, scheduling overhead, queue dynamics) solo aparecen bajo carga concurrente. Configura el dashboard de Grafana con las métricas de Prometheus de vLLM antes del primer despliegue de producción, no después del primer incidente.

Las métricas de serving son el lenguaje técnico que conecta los requisitos del producto (¿cuánto puede esperar el usuario?) con las decisiones de infraestructura (¿qué motor y qué hardware?). La sección de cierre sintetiza el capítulo y establece el servir correctamente como una competencia tan importante como el modelo mismo.

Módulo 8 – Capítulo 05 – Sección 06

Cierre: el serving eficiente es tan importante como el modelo mismo

La infraestructura de serving de LLMs no es el envoltorio de un modelo: es una de las decisiones técnicas más impactantes en la economía de un producto de IA. Un modelo de alta calidad desplegado con un motor de inferencia ineficiente puede tener peor experiencia de usuario y mayor costo operativo que un modelo de menor calidad correctamente optimizado. La diferencia entre Hugging Face Transformers en modo básico y vLLM con PagedAttention en el mismo hardware puede ser un factor de 3-5x en throughput y un factor de 2x en TTFT para cargas concurrentes, sin ninguna diferencia en el modelo, los pesos o la cuantización.

El ecosistema de serving de LLMs ha madurado al punto donde la elección del motor de inferencia es una decisión arquitectónica de primer orden que debe tomarse con el mismo rigor que la elección del modelo. La guía de selección presentada al inicio del capítulo — Ollama para prototipado local, vLLM para producción GPU de un modelo, Triton para múltiples tipos de modelos, TRT-LLM para máxima eficiencia en NVIDIA— captura la lógica de selección. Aplicarla correctamente requiere conocer los SLOs de latencia y throughput del producto antes de evaluar opciones de motor, y medir con métricas reales (TTFT P95, TBT P95, throughput agregado) en lugar de benchmarks sintéticos en aislamiento.

Las métricas de serving no son métricas de infraestructura sino métricas de producto. Un TTFT de 2 segundos en una aplicación de autocompletado de código destruye la experiencia de usuario independientemente de la calidad del modelo; un throughput insuficiente limita el número de usuarios que puede atender el producto, creando un techo de crecimiento técnico que solo puede resolverse con más hardware o con un motor más eficiente. El AI Engineer que diseña un sistema de serving de LLMs debe pensar simultáneamente en el modelo (calidad), el motor (throughput/latencia), el hardware (VRAM/costo) y los SLOs (TTFT P95, TBT P95, disponibilidad) como un sistema integrado donde la optimización de una dimensión afecta a las demás.

El capítulo siguiente aborda la especialización de los modelos mediante fine-tuning: cómo adaptar un modelo base a una tarea o dominio específico para mejorar la calidad en la tarea sin incrementar el costo de inferencia. Los capítulos de hardware (4) y serving (5) que acabas de completar son el contexto necesario para entender las restricciones de fine-tuning: QLoRA requiere al menos 8 GB de VRAM, Axolotl con DeepSpeed multi-GPU requiere conocer el hardware disponible en la nube, y el fine-tuned model debe eventualmente desplegarse en el motor de serving que ya tienes configurado.

Idea central

El triángulo de serving es calidad del modelo, latencia de respuesta y costo por token: siempre se puede mejorar uno de los tres a costa de los otros, y la decisión correcta depende exclusivamente de las prioridades del producto.

"A distributed system is one in which the failure of a computer you didn't even know existed can render your own computer unusable." — Leslie Lamport, Turing Award laureate, recordando que en sistemas de serving de LLMs, la cadena de dependencias (red, GPU driver, runtime, modelo) es tan frágil como su eslabón más débil.